中山大學
SUN YAT-SEN UNIVERSITY



公园信息管理与分析系统

数据结构与算法大作业汇报

成员：艾克散，蔡嘉谦，刘溢琛，来孜娜

汇报时间：2025.12.31

提纲

CONTENTS

1 任务背景

2 数据获取与预处理

3 公园查询模块

4 公园面积排序

5 计算最短路径

6 代码运行展示

7 问题与反思



01

来孜娜

数据获取，公园查询模块，实现按编号查询，按类型查询；

02

蔡嘉谦

实现按面积范围查询，整理测试用的公园数据；调查任务背景，并总结问题和反思；

03

刘溢琛

选择两种排序方式，对所有公园按面积进行排序；

04

艾克散

使用Dijkstra算法计算最短路径，输出路径序列与总距离，且能处理异常数据；



1

任务背景

任务背景

项目必要性：

- **城市公园数量快速增长**

近年来各地大力推进“公园城市建设”。
公园类型多样化，信息复杂度显著上升。

- **信息碎片化问题突出**

公园数据分散在不同平台，缺乏统一入口。
用户难以快速、准确地获取所需信息

- **现有工具功能有限**

一般地图应用侧重导航，无法支持按面积排序、按类型批量查询、基于多个条件组合筛选等需求。

任务可行性：

- **具备技术基础**

所需核心知识（结构体、vector、排序算法、图的存储与Dijkstra最短路径算法）均已在《数据结构与算法》课程中系统学习。

- **数据规模适中**

假设城市公园数量在几十个，数据量小、结构清晰，适合用内存中的线性表和邻接矩阵/表高效处理。

- **功能模块分工清晰**

系统可划分为：数据管理、查询、排序、路径计算等独立模块，便于分工协作与逐步开发，各模块之间接口明确，易于测试与集成。



2

数据获取与预处理

1. 数据获取与预处理

数据源：

	A	B	C	D	E	F
1	编号	公园名称	类型	面积（km	经纬度坐标	
2	1	白云山风	自然风景	20.98	(23.1847,	113.2972)
3	2	越秀公园	综合公园	6.82	(23.1393,	113.2714)
4	3	广州动物	动物园	2.31	(23.1410,	113.3084)
5	4	珠江公园	街心公园	0.28	(23.1235,	113.3306)
6	5	天河公园	社区公园	7.03	(23.1267,	113.3652)
7	6	海珠湖公	湿地公园	2.25	(23.0695,	113.3278)
8	7	荔湾湖公	街心公园	0.67	(23.1248,	113.2415)
9	8	大夫山森	森林公园	8	(22.9512,	113.3476)
10	9	华南植物	植物园	3.33	(23.1925,	113.3701)
11	10	广州塔公	主题公园	0.45	(23.1064,	113.3245)
12	11	流花湖公	综合公园	0.54	(23.1386,	113.2538)
13	12	东山湖公	街心公园	0.33	(23.1242,	113.2943)
14	13	晓港公园	社区公园	0.32	(23.0968,	113.2784)
15	14	黄埔公园	综合公园	1.02	(23.1028,	113.4346)
16	15	大学城中	湿地公园	1.85	(23.0593,	113.3942)
17	16	二沙岛体	体育公园	0.28	(23.1135,	113.3089)
18	17	广州科学	科技公园	3.5	(23.1657,	113.4458)
19	18	麓湖公园	自然风景	2.5	(23.1501,	113.2843)
20	19	番禺儿童	儿童公园	0.42	(22.9395,	113.3648)
21	20	南沙湿地	湿地公园	25	(22.6012,	113.6345)

Excel表格数据

↓

C++结构体Park定义（五个成员变量）

↓

Vector容器存储（动态数组）

```
struct Park {
    int id;
    string name;
    string type;
    double area;
    string location;
};
```

数据结构定义
名称
编号
类型
面积
坐标

```
vector<Park> parks = {
    {1, "白云山风景区", "自然风景区", 20.98, "(23.1847° N, 113.2972° E)"},
    {2, "越秀公园", "综合公园", 6.82, "(23.1393° N, 113.2714° E)"},
    {3, "广州动物园", "动物园", 2.31, "(23.1410° N, 113.3084° E)"},
    {4, "珠江公园", "街心公园", 0.28, "(23.1235° N, 113.3306° E)"},
    {5, "天河公园", "社区公园", 7.03, "(23.1267° N, 113.3652° E)"},
    {6, "海珠湖公园", "湿地公园", 2.25, "(23.0695° N, 113.3278° E)"},
    {7, "荔湾湖公园", "街心公园", 0.67, "(23.1248° N, 113.2415° E)"},
    {8, "大夫山森林公园", "森林公园", 8.0, "(22.9512° N, 113.3476° E)"},
    {9, "华南植物公园", "植物园", 3.33, "(23.1925° N, 113.3701° E)"},
    {10, "广州塔公园", "主题公园", 0.45, "(23.1064° N, 113.3245° E)"},
    {11, "流花湖公园", "综合公园", 0.54, "(23.1386° N, 113.2538° E)"},
    {12, "东山湖公园", "街心公园", 0.33, "(23.1242° N, 113.2943° E)"},
    {13, "晓港公园", "社区公园", 0.32, "(23.0968° N, 113.2784° E)"},
    {14, "黄埔公园", "综合公园", 1.02, "(23.1028° N, 113.4346° E)"},
    {15, "大学城中心湖公园", "湿地公园", 1.85, "(23.0593° N, 113.3942° E)"},
    {16, "二沙岛体育公园", "体育公园", 0.28, "(23.1135° N, 113.3089° E)"},
    {17, "广州科学城公园", "科技公园", 3.5, "(23.1657° N, 113.4458° E)"},
    {18, "麓湖公园", "自然风景公园", 2.5, "(23.1501° N, 113.2843° E)"},
    {19, "番禺儿童公园", "儿童公园", 0.42, "(22.9395° N, 113.3648° E)"},
    {20, "南沙湿地公园", "湿地公园", 25.0, "(22.6012° N, 113.6345° E)"}
};
```

数据初始化

1. 数据获取与预处理

设计选择

✓ **结构体而非类**
- 只需数据存储
- 内存连续高效
- 实现简单直接

✓ **vector而非数组**
- 动态大小
- 自带迭代器
- 易于维护扩展

✓ **全局变量存储**
- 避免参数传递
- 模块共享方便
- 调用简洁

设计理念

•基于结构体的高效存储

•利用vector的动态管理

•面向未来的可扩展架构

数据质量

✓ **完整性**
- 20条记录完整
- 5个字段无缺失
- id 1-20不重复

✓ **一致性**
- 面积单位统一(km²)
- 坐标格式规范
- 类型分类清晰

✓ **可扩展性**
- 支持新增公园
- 支持添加字段
- 便于功能扩展



3

公园查询模块

2. 公园查询模块 → 按编号查询

```
Park* Search_By_Id(int id) {
    for (int i = 0; i < parks.size(); i++) {
        if (parks[i].id == id) {
            return &parks[i];
        }
    }
    return nullptr;
}
```

线性查找适合小规模数据

返回指针提升效率

空指针用于异常处理

核心函数：Park* Search_By_Id(int id)

时间复杂度：O(n)
空间复杂度：O(1)

2. 公园查询模块 按类型查询

```
vector<Park> Search_By_Type(string type) {
    vector<Park> result;
    for (int i = 0; i < parks.size(); i++) {
        if (parks[i].type == type) {
            result.push_back(parks[i]);
        }
    }
    return result;
}
```

返回vector支持多结果

支持批量查询

空vector表示无结果

核心函数：vector<Park> Search_By_Type(string type)

时间复杂度：O(n)
空间复杂度：O(m)(m为匹配数量)

2. 公园查询模块 → 按面积范围查询

```
vector<Park> Search_By_Area_Range(double min_area, double max_area) {
    vector<Park> result;
    for (int i = 0; i < parks.size(); i++) {
        if (parks[i].area >= min_area && parks[i].area <= max_area) {
            result.push_back(parks[i]);
        }
    }
    return result;
}
```

遍历所有公园筛选面积

支持批量查询

空vector表示无结果

核心函数：
vector<Park> Search_By_Area_Range(double min_area, double max_area)

时间复杂度：O(n)
空间复杂度：O(m)(m为匹配数量)

2. 公园查询模块 → 格式化输出

```
void Print_Park_Info(const Park* park) {
    if (park == nullptr) {
        cout << "未找到该公园。\n";
        return;
    }
    cout << "\n编号: " << park->id << endl;
    cout << "名称: " << park->name << endl;
    cout << "类型: " << park->type << endl;
    cout << "面积: " << park->area << " km^2" << endl;
    cout << "位置: " << park->location << endl;
}
```

→ 输出单个公园

```
void Print_Multiple_Parks(const vector<Park>& list) {
    if (list.size() == 0) {
        cout << "未找到符合条件的公园。\n";
        return;
    }

    cout << "\n共找到 " << list.size() << " 个公园: \n";
    for (int i = 0; i < list.size(); i++) {
        Print_Park_Info(&list[i]);
        cout << "--------------------" << endl;
    }
}
```

→ 输出多个公园

设计原则：一致性，完整性，友好性，健壮性，扩展性

设计特点	实现方式	用户价值
信息结构化	固定5字段顺序	信息清晰，易于阅读
异常友好	明确的“未找到”提示	避免用户困惑
视觉层次	分割线+统计头部	提升可读性
代码复用	批量调用单个输出	维护简单，风格统一
单位明确	面积标注“km²”	避免理解歧义

2. 公园查询模块 异常处理

编号异常处理

```
Park* Search_By_Id(int id) {
    for (int i = 0; i < parks.size(); i++) {
        if (parks[i].id == id) {
            return &parks[i];
        }
    }
    return nullptr;
}

void Print_Park_Info(const Park* park) {
    if (park == nullptr) {
        cout << "未找到该公园。\n";
        return;
    }
    cout << "\n编号：" << park->id << endl;
    cout << "名称：" << park->name << endl;
    cout << "类型：" << park->type << endl;
    cout << "面积：" << park->area << " km^2" << endl;
    cout << "位置：" << park->location << endl;
}
```

查询函数：编号不存在时返回空指针

标识异常状态

输出函数：检测并处理空指针

编号异常检测点

核心思想：前置验证，无效即返，明确提示

1.异常场景： 用户输入的公园编号不存在于系统中。

2.检测机制：

Search_By_Id() 函数遍历后未找到匹配项，返回 nullptr；

Print_Park_Info() 在输出前检测指针有效性

3.处理逻辑：

查询失败 → 返回nullptr → 检测到空指针 → 提示用户 → 安全返回

2. 公园查询模块 异常处理

类型异常处理

```
vector<Park> Search_By_Type(string type) {
    vector<Park> result;
    for (int i = 0; i < parks.size(); i++) {
        if (parks[i].type == type) {
            result.push_back(parks[i]);
        }
    }
    return result;
}
```

查询函数：返回匹配结果集

空vector表示无匹配

```
void Print_Multiple_Parks(const vector<Park>& list) {
    if (list.size() == 0) {
        cout << "未找到符合条件的公园。\n";
        return;
    }

    cout << "\n共找到 " << list.size() << " 个公园：\n";
    for (int i = 0; i < list.size(); i++) {
        Print_Park_Info(&list[i]);
        cout << "--------------------" << endl;
    }
}
```

输出函数：检测空结果集

类型异常检测点

核心思想：前置验证，无效即返，明确提示

1.异常场景： 用户输入的公园类型在系统中无匹配记录。

2.检测机制：

Search_By_Type() 函数返回空的 vector；

Print_Multiple_Parks() 在输出前检测结果集大小

3.处理逻辑：

查询无匹配 → 返回空vector → 检测到size()==0 → 提示用户 → 终止输出流程 **（面积范围异常时处理类似）**



4

公园面积排序

3. 公园面积排序 冒泡排序

核心函数：
void BubbleSort(vector<Park>& parks)

```
void BubbleSort(vector<Park>& parks) {
    int n = parks.size();
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - 1 - i; ++j) {
            if (parks[j].area > parks[j + 1].area) {
                Park temp = parks[j];
                parks[j] = parks[j + 1];
                parks[j + 1] = temp;
            }
        }
    }
}
```

外层循环：

控制排序轮数共n-1轮，

每轮将最大的元素"泡"到末尾

内层循环：

每轮遍历未排序的元素，相邻比较交换

核心判断：

仅当前面>后面时交换，面积相等不交换

3. 公园面积排序 快速排序

分区函数：

快速排序的核心逻辑，将大数组拆成小数组，实现高效排序：

```
int Partition(vector<Park>& parks, int low, int high) {
    double pivot = parks[high].area;
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (parks[j].area <= pivot) {
            i++;
            Park temp = parks[i];
            parks[i] = parks[j];
            parks[j] = temp;
        }
    }
    Park temp = parks[i + 1];
    parks[i + 1] = parks[high];
    parks[high] = temp;
    return i + 1;
}
```

选区间最后一个元素作为基准值

核心判断：面积≤基准的元素放到左侧

基准值归位，并返回基准值的最终下标，作为下一次分区的分割点

快排函数：

提供高效的排序方案，平均速度远快于冒泡排序：

```
void QuickSort(vector<Park>& parks, int low, int high) {
    if (low < high) {
        int pivotIdx = Partition(parks, low, high);
        QuickSort(parks, low, pivotIdx - 1);
        QuickSort(parks, pivotIdx + 1, high);
    }
}
```

递归排序左/右侧小数组

分区并获取基准下标

3. 公园面积排序 结果打印

排序前打印：

专门打印排序前原始公园数据，避免重复代码

```
void PrintBeforeSort() {
    cout << "\n---------- 排序前原始顺序 ----------\n";
    for (int i = 0; i < parks.size(); i++) {
        cout << "编号: " << parks[i].id
            << " | 名称: " << parks[i].name
            << " | 面积: " << parks[i].area << " km^2\n";
    }
}
```

代码复用，不用在每个排序选项中重复写打印循环；

让用户清晰看到排序前顺序，便于对比

排序部分子菜单：

显示排序方法选择界面，引导用户操作

```
void Show_Sort_Submenu() {
    cout << "\n---------- 排序方法选择 ----------\n";
    cout << "1. 冒泡排序\n";
    cout << "2. 快速排序\n";
    cout << "0. 返回主菜单\n";
    cout << "--------------------------------\n";
    cout << "请选择（0-2）: ";
}
```

列出2个排序选项和0=返回主菜单，交互引导；

逻辑拆分，和主菜单分离，代码结构更清晰

3. 公园面积排序 结果打印

```
else if (choice == 4) {
    int sort_choice;
    while (true) {
        Show_Sort_Submenu();
        cin >> sort_choice;
        vector<Park> parkCopy = parks;

        if (sort_choice == 1) {
            cout << "\n===== 选择1：冒泡排序 =====";
            PrintBeforeSort();
            BubbleSort(parkCopy);
            PrintSortedParks(parkCopy, "冒泡排序后");
            break;
        }
        else if (sort_choice == 2) {
            cout << "\n===== 选择2：快速排序 =====";
            PrintBeforeSort();
            QuickSort(parkCopy, 0, parkCopy.size() - 1);
            PrintSortedParks(parkCopy, "快速排序后");
            break;
        }
        else if (sort_choice == 0) {
            cout << "返回主菜单...\n";
            break;
        }
        else {
            cout << "输入错误！请选0-2之间的数字\n";
            cin.clear();
            cin.ignore(1000, '\n');
        }
    }
}
```

主函数排序功能：

按用户选择的编号调用对应排序方法，均显示排序前后结果

打印排序前原始数据

执行冒泡/快速排序

打印排序后结果

异常情况：

交互优化，用户未输入正确数字，返回错误提示

3. 公园面积排序 → 算法效率比较

```
---------- 排序前原始顺序 ----------
编号: 1 | 名称: 白云山风景区 | 面积: 20.98 km^2
编号: 2 | 名称: 越秀公园 | 面积: 6.82 km^2
编号: 3 | 名称: 广州动物园 | 面积: 2.31 km^2
编号: 4 | 名称: 珠江公园 | 面积: 0.28 km^2
编号: 5 | 名称: 天河公园 | 面积: 7.03 km^2
编号: 6 | 名称: 海珠湖公园 | 面积: 2.25 km^2
编号: 7 | 名称: 荔湾湖公园 | 面积: 0.67 km^2
编号: 8 | 名称: 大夫山森林公园 | 面积: 8 km^2
编号: 9 | 名称: 华南植物公园 | 面积: 3.33 km^2
编号: 10 | 名称: 广州塔公园 | 面积: 0.45 km^2
编号: 11 | 名称: 流花湖公园 | 面积: 0.54 km^2
编号: 12 | 名称: 东山湖公园 | 面积: 0.33 km^2
编号: 13 | 名称: 晓港公园 | 面积: 0.32 km^2
编号: 14 | 名称: 黄埔公园 | 面积: 1.02 km^2
编号: 15 | 名称: 大学城中心湖公园 | 面积: 1.85 km^2
编号: 16 | 名称: 二沙岛体育公园 | 面积: 0.28 km^2
编号: 17 | 名称: 广州科学城公园 | 面积: 3.5 km^2
编号: 18 | 名称: 麓湖公园 | 面积: 2.5 km^2
编号: 19 | 名称: 番禺儿童公园 | 面积: 0.42 km^2
编号: 20 | 名称: 南沙湿地公园 | 面积: 25 km^2
```

→ 排序前结果

```
---------- 冒泡排序后 ----------
编号: 4 | 名称: 珠江公园 | 面积: 0.28 km^2
编号: 16 | 名称: 二沙岛体育公园 | 面积: 0.28 km^2
编号: 13 | 名称: 晓港公园 | 面积: 0.32 km^2
编号: 12 | 名称: 东山湖公园 | 面积: 0.33 km^2
编号: 19 | 名称: 番禺儿童公园 | 面积: 0.42 km^2
编号: 10 | 名称: 广州塔公园 | 面积: 0.45 km^2
编号: 11 | 名称: 流花湖公园 | 面积: 0.54 km^2
编号: 7 | 名称: 荔湾湖公园 | 面积: 0.67 km^2
编号: 14 | 名称: 黄埔公园 | 面积: 1.02 km^2
编号: 15 | 名称: 大学城中心湖公园 | 面积: 1.85 km^2
编号: 6 | 名称: 海珠湖公园 | 面积: 2.25 km^2
编号: 3 | 名称: 广州动物园 | 面积: 2.31 km^2
编号: 18 | 名称: 麓湖公园 | 面积: 2.5 km^2
编号: 9 | 名称: 华南植物公园 | 面积: 3.33 km^2
编号: 17 | 名称: 广州科学城公园 | 面积: 3.5 km^2
编号: 2 | 名称: 越秀公园 | 面积: 6.82 km^2
编号: 5 | 名称: 天河公园 | 面积: 7.03 km^2
编号: 8 | 名称: 大夫山森林公园 | 面积: 8 km^2
编号: 1 | 名称: 白云山风景区 | 面积: 20.98 km^2
编号: 20 | 名称: 南沙湿地公园 | 面积: 25 km^2
```

→ 排序后结果

	冒泡排序	快速排序
运行时间	快（本实验中 20 个公园，差异不明显）	快（和冒泡排序差距不大，肉眼难以感知）
时间复杂度（本实验为平均情况）	O (n^2)	O (nlogn)

分析：

本实验场景中，小数据量下（如 20 个公园），两种算法运行时间差异不明显，用冒泡排序即可，逻辑简单，容易实现和调试；大数据量下（如 10000 个数据），快速排序运行时间远少于冒泡排序，效率优势显著。



5

计算最短路径

4. 计算最短路径

写函数前必要的参数定义

```
#define INF 1000000000
```

定义“∞”为INF其表示两个公园之间不存在直接的通路

```
const double Park_Dist_Graph[20][20] = {
    {0,     5.21,   INF,    INF,    INF,    INF,    INF,    INF,    6.87,   INF,    INF,    INF,    INF,    INF,    INF,    INF,    8.92,   4.03,   INF,    INF},
    {5.21,  0,      INF,    3.42,   INF,    INF,    2.96,   INF,    INF,    INF,    1.85,   2.1,    INF,    INF,    INF,    INF,    INF,    3.88,   INF,    INF},
    {INF,   INF,    0,      2.78,   INF,    INF,    INF,    INF,    INF,    3.12,   INF,    2.35,   4.56,   INF,    INF,    INF,    INF,    INF,    INF,    INF},
    {INF,   3.42,   2.78,   0,      3.87,   INF,    INF,    INF,    INF,    1.98,   INF,    INF,    INF,    INF,    INF,    1.25,   INF,    INF,    INF,    INF},
    {INF,   INF,    INF,    3.87,   0,      INF,    INF,    INF,    4.1,    INF,    INF,    INF,    INF,    INF,    6.33,   INF,    7.21,   INF,    INF,    INF},
    {INF,   INF,    INF,    INF,    INF,    0,      INF,    7.89,   INF,    2.85,   INF,    INF,    3.02,   INF,    5.67,   INF,    INF,    INF,    8.44,   INF},
    {INF,   2.96,   INF,    INF,    INF,    INF,    0,      INF,    INF,    INF,    1.68,   INF,    INF,    9.21,   INF,    INF,    INF,    INF,    INF,    INF},
    {INF,   INF,    INF,    INF,    INF,    7.89,   INF,    0,      INF,    INF,    INF,    INF,    INF,    INF,    6.55,   INF,    INF,    INF,    4.33,   INF},
    {6.87,  INF,    INF,    INF,    4.1,    INF,    INF,    INF,    0,      INF,    INF,    INF,    INF,    INF,    INF,    INF,    5.44,   5.12,   INF,    INF},
    {INF,   INF,    3.12,   1.98,   INF,    2.85,   INF,    INF,    INF,    0,      INF,    INF,    2.2,    INF,    INF,    1.1,    INF,    INF,    INF,    INF},
    {INF,   1.85,   INF,    INF,    INF,    INF,    1.68,   INF,    INF,    INF,    0,      INF,    INF,    8.76,   INF,    INF,    INF,    INF,    INF,    INF},
    {INF,   2.1,    2.35,   INF,    INF,    INF,    INF,    INF,    INF,    INF,    INF,    0,      2.65,   INF,    INF,    INF,    INF,    INF,    INF,    INF},
    {INF,   INF,    4.56,   INF,    INF,    3.02,   INF,    INF,    INF,    2.2,    INF,    2.65,   0,      INF,    INF,    INF,    INF,    INF,    6.78,   INF},
    {INF,   INF,    INF,    INF,    INF,    INF,    9.21,   INF,    INF,    INF,    8.76,   INF,    INF,    0,      INF,    INF,    3.21,   INF,    INF,    INF},
    {INF,   INF,    INF,    INF,    6.33,   5.67,   INF,    6.55,   INF,    INF,    INF,    INF,    INF,    INF,    0,      INF,    INF,    INF,    INF,    12.34},
    {INF,   INF,    INF,    1.25,   INF,    INF,    INF,    INF,    INF,    1.1,    INF,    INF,    INF,    INF,    INF,    0,      INF,    INF,    INF,    INF},
    {8.92,  INF,    INF,    INF,    7.21,   INF,    INF,    INF,    5.44,   INF,    INF,    INF,    INF,    3.21,   INF,    INF,    0,      INF,    INF,    15.67},
    {4.03,  3.88,   INF,    INF,    INF,    INF,    INF,    INF,    5.12,   INF,    INF,    INF,    INF,    INF,    INF,    INF,    INF,    0,      INF,    INF},
    {INF,   INF,    INF,    INF,    INF,    8.44,   INF,    4.33,   INF,    INF,    INF,    INF,    6.78,   INF,    INF,    INF,    INF,    INF,    0,      18.22},
    {INF,   INF,    INF,    INF,    INF,    INF,    INF,    INF,    INF,    INF,    INF,    INF,    INF,    INF,    12.34,  INF,    15.67,  INF,    18.22,  0}
};
```

使用邻接矩阵表示两个公园之间的距离(前提是二者之间可以直接到达)

```
const int park_total_number = 20;
```

定义公园数为常数不可以被修改

4. 计算最短路径

写函数前必要的参数定义

因为后续发现使用邻接矩阵表示过于麻烦，所以此处换为了使用下三角矩阵的方式去存储两个公园之间的距离

```
const double Park_Dist_Graph[20][20] = {
    {0,    5.21,  INF,   INF,   INF,   INF,   INF,   INF,   6.87,  INF,   INF,   INF,   INF,   INF,   INF,   INF,   8.92,  4.03,  INF,   INF},
    {5.21,  0,    INF,   3.42,  INF,   INF,   2.96,  INF,   INF,   INF,   1.85,  2.1,   INF,   INF,   INF,   INF,   INF,   3.88,  INF,   INF},
    {INF,   INF,   0,    2.78,  INF,   INF,   INF,   INF,   INF,   3.12,  INF,   2.35,  4.56,  INF,   INF,   INF,   INF,   INF,   INF,   INF},
    {INF,   3.42,  2.78,  0,    3.87,  INF,   INF,   INF,   INF,   1.98,  INF,   INF,   INF,   INF,   INF,   1.25,  INF,   INF,   INF,   INF},
    {INF,   INF,   INF,   3.87,  0,    INF,   INF,   INF,   4.1,   INF,   INF,   INF,   INF,   INF,   6.33,  INF,   7.21,  INF,   INF,   INF},
    {INF,   INF,   INF,   INF,   INF,   0,    INF,   7.89,  INF,   2.85,  INF,   INF,   3.02,  INF,   5.67,  INF,   INF,   INF,   8.44,  INF},
    {INF,   2.96,  INF,   INF,   INF,   INF,   0,    INF,   INF,   INF,   1.68,  INF,   INF,   9.21,  INF,   INF,   INF,   INF,   INF,   INF},
    {INF,   INF,   INF,   INF,   INF,   7.89,  INF,   0,    INF,   INF,   INF,   INF,   INF,   INF,   6.55,  INF,   INF,   INF,   4.33,  INF},
    {6.87,  INF,   INF,   INF,   4.1,   INF,   INF,   INF,   0,    INF,   INF,   INF,   INF,   INF,   INF,   INF,   5.44,  5.12,  INF,   INF},
    {INF,   INF,   3.12,  1.98,  INF,   2.85,  INF,   INF,   INF,   0,    INF,   INF,   2.2,   INF,   INF,   1.1,   INF,   INF,   INF,   INF},
    {INF,   1.85,  INF,   INF,   INF,   INF,   1.68,  INF,   INF,   INF,   0,    INF,   INF,   8.76,  INF,   INF,   INF,   INF,   INF,   INF},
    {INF,   2.1,   2.35,  INF,   INF,   INF,   INF,   INF,   INF,   INF,   INF,   0,    2.65,  INF,   INF,   INF,   INF,   INF,   INF,   INF},
    {INF,   INF,   4.56,  INF,   INF,   3.02,  INF,   INF,   INF,   2.2,   INF,   2.65,  0,    INF,   INF,   INF,   INF,   INF,   6.78,  INF},
    {INF,   INF,   INF,   INF,   INF,   INF,   9.21,  INF,   INF,   INF,   8.76,  INF,   INF,   0,    INF,   INF,   3.21,  INF,   INF,   INF},
    {INF,   INF,   INF,   INF,   6.33,  5.67,  INF,   6.55,  INF,   INF,   INF,   INF,   INF,   INF,   0,    INF,   INF,   INF,   INF,   12.34},
    {INF,   INF,   INF,   1.25,  INF,   INF,   INF,   INF,   INF,   1.1,   INF,   INF,   INF,   INF,   INF,   0,    INF,   INF,   INF,   INF},
    {8.92,  INF,   INF,   INF,   7.21,  INF,   INF,   INF,   5.44,  INF,   INF,   INF,   INF,   3.21,  INF,   INF,   0,    INF,   INF,   15.67},
    {4.03,  3.88,  INF,   INF,   INF,   INF,   INF,   INF,   5.12,  INF,   INF,   INF,   INF,   INF,   INF,   INF,   INF,   0,    INF,   INF},
    {INF,   INF,   INF,   INF,   INF,   8.44,  INF,   4.33,  INF,   INF,   INF,   INF,   6.78,  INF,   INF,   INF,   INF,   INF,   0,    18.22},
    {INF,   INF,   INF,   INF,   INF,   INF,   INF,   INF,   INF,   INF,   INF,   INF,   INF,   INF,   12.34, INF,   15.67, INF,   18.22, 0}
};
```

邻接矩阵表示

对角线均设置为0

```
for (int i = 0; i < 20; i++)
    for (int j = 0; j < 20; j++)
        Park_Dist_Graph[i][j] = (i == j) ? 0 : INF;
```

```
Park_Dist_Graph[1][0] = 5.21;
Park_Dist_Graph[8][0] = 6.87;
Park_Dist_Graph[16][0] = 8.92;
Park_Dist_Graph[17][0] = 4.03;
Park_Dist_Graph[3][1] = 3.42;
Park_Dist_Graph[6][1] = 2.96;
Park_Dist_Graph[10][1] = 1.85;
Park_Dist_Graph[11][1] = 2.1;
Park_Dist_Graph[17][1] = 3.88;
Park_Dist_Graph[3][2] = 2.78;
Park_Dist_Graph[9][2] = 3.12;
Park_Dist_Graph[11][2] = 2.35;
Park_Dist_Graph[12][2] = 4.56;
Park_Dist_Graph[4][3] = 3.87;
Park_Dist_Graph[9][3] = 1.98;
Park_Dist_Graph[15][3] = 1.25;
Park_Dist_Graph[8][4] = 4.1;
Park_Dist_Graph[14][4] = 6.33;
Park_Dist_Graph[16][4] = 7.21;
Park_Dist_Graph[7][5] = 7.89;
Park_Dist_Graph[9][5] = 2.85;
Park_Dist_Graph[12][5] = 3.02;
Park_Dist_Graph[14][5] = 5.67;
Park_Dist_Graph[18][5] = 8.44;
Park_Dist_Graph[10][6] = 1.68;
Park_Dist_Graph[13][6] = 9.21;
Park_Dist_Graph[14][7] = 6.55;
Park_Dist_Graph[18][7] = 4.33;
Park_Dist_Graph[16][8] = 5.44;
Park_Dist_Graph[17][8] = 5.12;
Park_Dist_Graph[12][9] = 2.2;
Park_Dist_Graph[15][9] = 1.1;
Park_Dist_Graph[13][10] = 8.76;
Park_Dist_Graph[12][11] = 2.65;
Park_Dist_Graph[18][12] = 6.78;
Park_Dist_Graph[16][13] = 3.21;
Park_Dist_Graph[19][14] = 12.34;
Park_Dist_Graph[19][16] = 15.67;
Park_Dist_Graph[19][18] = 18.22;
```

```
for (int i = 0; i < 20; i++) {
    for (int j = 0; j < i; j++) {
        Park_Dist_Graph[j][i] = Park_Dist_Graph[i][j];
    }
}
```

下三角矩阵的方式去存储

4. 计算最短路径 Dijkstra算法

计算最短路径函数的形参设置：

```
bool Shortest_Path(int start_id, int end_id, vector<int>& path, double& total_dist)
```

起始公园的编号

终止公园的编号

所经过的路径

所经过的总距离

4. 计算最短路径 Dijkstra算法

case1：若起始与终止公园的编号相同

step1：输出的路径为该公园的编号，并且输出的总距离为0：

```
if (start_id == end_id) {
    path.push_back(start_id);
    total_dist = 0;
    return true;
}
```

```
#include <vector>
```

4. 计算最短路径 Dijkstra算法

case2：若起始与终止公园的编号不相同

step1：初始化

```
int start = start_id - 1;
int end = end_id - 1;
```

编号-1防止数组越界的风险

```
double dist[park_total_number];
```

存储start到第i个点的最短路径

```
bool visited[park_total_number];
```

存储已经被访问过的结点

```
int prev_number[park_total_number];
```

存储到某点最短路径经过的结点

```
for (int i = 0; i < park_total_number; i++) {
    dist[i] = Park_Dist_Graph[start][i];
    visited[i] = false;
    prev_number[i] = (dist[i] < INF && i != start) ? start : -1;
}
dist[start] = 0;
visited[start] = true;
```

初始化与start相连的节点的路径，且将start设置为已经访问过的结点

4. 计算最短路径 Dijkstra算法

case2：若起始与终止公园的编号不相同

step2：编写主体函数

```
for (int i = 1; i < park_total_number; i++) {
    double min_dist = INF;
    int min_id = -1;

    for (int j = 0; j < park_total_number; j++) {
        if (!visited[j] && dist[j] < min_dist) {
            min_dist = dist[j];
            min_id = j;
        }
    }

    if (min_id == -1) break;
    visited[min_id] = true;

    for (int j = 0; j < park_total_number; j++) {
        if (!visited[j] && Park_Dist_Graph[min_id][j] < INF && dist[min_id] + Park_Dist_Graph[min_id][j] < dist[j]) {
            dist[j] = dist[min_id] + Park_Dist_Graph[min_id][j];
            prev_number[j] = min_id;
        }
    }
}

if (dist[end] >= INF) {
    return false;
}
```

找到当前的最短路径

dijkstra算法的灵魂

更新当前的min_id到与其相连接的结点的距离

4. 计算最短路径 Dijkstra算法

case2：若起始与终止公园的编号不相同

step3：输出计算最短路径时所经过的路径

```
int current = end;
while (current != -1) {
    path.push_back(current + 1);
    current = prev_number[current];
}
```

此处得到的path是逆着存储所经过的路径的，因为在一开始我们是使current=end

```
reverse(path.begin(), path.end());
total_dist = dist[end];
return true;
```

反转我们得到的路径此处必须写相对应的主函数才可以调用reverse函数

```
#include <algorithm>
```

4. 计算最短路径 编写打印函数

```
void Print_Shortest_Path(int start_id, int end_id, const vector<int>& path, double total_dist) {
    cout << "===== 最短路径查询结果 =====" << endl;
    Park* start_park = Search_By_Id(start_id);
    Park* end_park = Search_By_Id(end_id);
    if (start_park != nullptr) {
        cout << "起点的公园名称：" << start_park->name << " 其编号：" << start_park->id << endl;
    }
    if (end_park != nullptr) {
        cout << "终点的公园名称：" << end_park->name << " 其编号：" << end_park->id << endl;
    }
    cout << "二者的总距离为：" << total_dist << " 千米" << endl;
    cout << "经过的路径的顺序为：";
    for (int i = 0; i < path.size(); i++) {
        Park* p = Search_By_Id(path[i]);
        if (p != nullptr) {
            cout << p->name;
        }
        if (i < path.size() - 1) {
            cout << "->";
        }
    }
    cout << endl << "============================" << endl;
}
```

调用用编号查询的函数

若此处不写限制的函数则在最后一个编号的后面会多输出一个->

4. 计算最短路径

编写读取输入函数

```
int Enter_Number() {
    int number;
    cin >> number;
    while (number > 20 || number < 1) {
        cout << "============================" << endl;
        cout << "您输入的值的范围有误！公园编号的范围是1-20！" << endl;
        cout << "============================" << endl << endl;
        cout << "请重新输入该公园的编号:";
        cin >> number;
    }
    return number;
}
```

可以有效的防止用户输入超出范围的编号

4. 计算最短路径 编写主函数

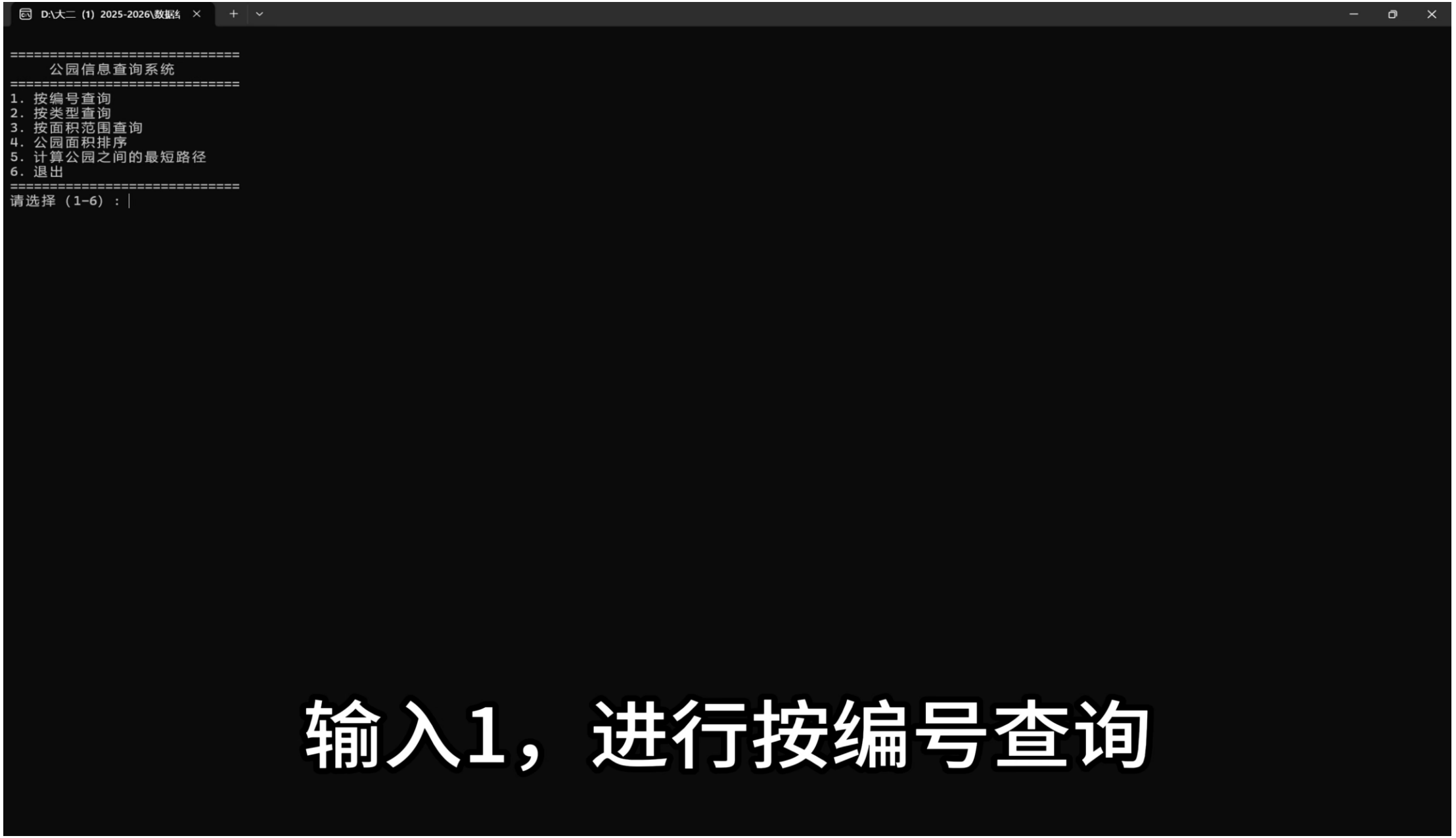
```
else if (choice == 5) {
    int start_id, end_id;
    cout << "请输入起始公园编号：";
    start_id = Enter_Number();
    cout << "请输入终止公园编号：";
    end_id = Enter_Number();
    vector<int> path;
    double total_dist;
    bool success = Shortest_Path(start_id, end_id, path, total_dist);
    if (success) {
        Print_Shortest_Path(start_id, end_id, path, total_dist);
    }
    else {
        cout << "============================" << endl;
        cout << "两个公园之间不存在可达路径！";
        cout << endl << "============================" << endl;
    }
}
```

此处即调用我们编写的计算最短路径的函数



6

代码运行展示


D:\大二 (1) 2025-2026\数据结
==============================
公园信息查询系统
==============================
1. 按编号查询
2. 按类型查询
3. 按面积范围查询
4. 公园面积排序
5. 计算公园之间的最短路径
6. 退出
==============================
请选择（1-6）：
输入1，进行按编号查询



7

反思与问题

反思与问题

技术实现层面：

- **排序算法选择：**

虽然实现了冒泡排序和快速排序，但在小规模数据（20个公园）下性能差异不明显。若未来数据量激增，需切换至更高效算法。

- **图存储结构局限性：**

初期采用邻接矩阵，后改为下三角矩阵节省空间，但仍存在稀疏图存储浪费问题。未来可改用邻接表提升空间效率。

系统设计层面：

- **全局变量依赖：**

为简化模块调用使用了全局 vector，虽便于开发，但降低了代码可测试性与模块独立性。

- **查询功能耦合度高：**

三种查询（ID/类型/面积）逻辑相似但代码重复。可抽象为统一过滤接口，提升可维护性

谢谢聆听！